Listing 5.4 Testing constraints

```
function validateConstraints(text, minWords, maxWords, requiredWords) {
  const words = text.split(/\s+/);
  return (
    words.length >= minWords &&
    words.length <= maxWords &&
    requiredWords.every((word) =>
      text.toLowerCase().includes(word.toLowerCase())
    )
  );
}
test("generated text meets constraints", async () => {
generateText.mockResolvedValue({
  text: "Regular exercise significantly improves
health and fitness by boosting energy, enhancing mood,
and reducing the risk of many chronic diseases."
});

  const { text } = await generateText({
    model: "fakeModel",
    prompt:
      "Summarize the benefits of exercise in 20-30 words.
Include 'health' and 'fitness'.",
  });

  console.log("Word count:", text.split(/\s+/).length);
  expect(validateConstraints(text, 20, 30, ["health", "fitness"])).
toBe(true);
});
```

Checks if the generated text meets specific word count requirements and includes required words

The prompt is used to instruct the AI model on the expected output, including constraints on word count and the necessity of specific keywords.

Verifies whether the generated text adheres to the defined constraints, ensuring that the test case validates the functionality correctly

In the example, the `validateConstraints` function is defined to check whether the generated text meets specific conditions: it must contain a certain number of words (`minWords` and `maxWords`) and include certain required words (`requiredWords`). The `validateConstraints` function splits the text into words and then verifies that the word count falls within the specified range and that all required words are present in the text, regardless of case.

Constrained validation is useful when ensuring that AI outputs follow specific rules (provided in the prompt), making it ideal for scenarios where the generated content must adhere to strict guidelines, such as summaries, reports, or any content requiring specific terminology or structure. However, as you need to preconfigure those rules beforehand, it quickly becomes unwieldy and hard to keep track of unless you create a system that configures the test criteria on the fly.

The preceding list of strategies is not exhaustive, but they can help you catch potential issues early and also ensure that the AI outputs meet specific criteria and deliver consistent, high-quality results.